

Exploring Korf's Method of Solving a Rubik's Cube with IDA* Search using Pattern Databases

Summer 2022 Group Project Report by Group 42

Jack Manuel

, Keene Upathamp

, and Tara H. Kazemi

Introduction

A scrambled 3x3 Rubik's Cube can have over 43 quintillion permutations, which makes it nearly impossible for a computer to solve with a brute force algorithm. Applying pathfinding AI algorithms requires iterating through very large trees, with a branching factor of over 12. We attempted to solve the Rubik's Cube by applying (Iterative Deepening A* Search) IDA* with pattern databases. IDA* search allows us to limit the scope of the search space and pattern databases allow us to precalculate and store a minimum move heuristic for several subsets of the cube. Although there is a significant overhead cost in implementing pattern databases, this strategy does lead to an optimal solution every time.

Iterative Deepening with Depth First Search iteratively traverses the search tree, visiting the nodes in a depth-limiting fashion, that is once the maximum depth is fully explored, the search restarts at the root with an increased maximum depth. A* search intends to find the shortest, or least costly, path to the goal state by exploring more promising paths first. The decision to traverse a path or choose another is based on a set of heuristics and cost calculations that aims to minimize $f(n)$.

$f(n) = g(n) + h(n)$ where $g(n)$ is the cost of the path from the starting state to the current state, and $h(n)$ is a heuristic value based on the cost of the cheapest path from the current state to the goal state (Korf & Felner, 2002). If $h(n)$ is admissible, then A* will find the shortest possible path. When the cost of extending a given path becomes too large or when the goal state has been reached, the search stops.

Implementation

We implemented Korf's algorithm which is proven to solve any scrambled cube optimally in at most 20 moves by searching for goal states using Iterative-Deepening Depth First Search combined with A* search. The algorithm uses pattern databases to store several different precomputed heuristics.

IDA* Search

We decided to implement a non-recursive version of IDA* to avoid the extra overhead of the function call stack over billions of calls. The search will never return failure, since the algorithm only takes in solvable cubes as input. It will search until an optimal solution is found.

Algorithm 1: IDA* Search

```

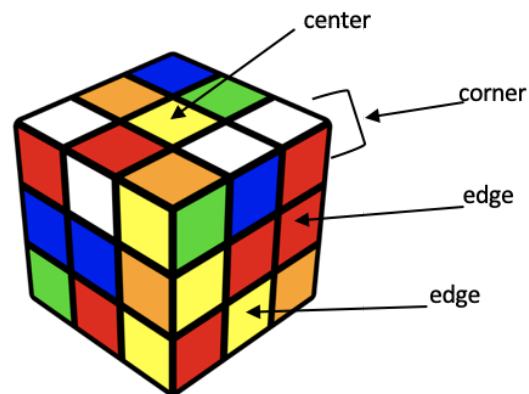
root=initial node
goal=final node
function IDAstar(){
    threshold=heuristic(Start)
    while(not empty) do
        temp = DFSearch(Start,0,threshold)
        if(temp == FOUND):
            return TRUE
        if (temp ==  $\infty$ ):
            return FALSE
        threshold=temp
    }//end driver function

function DFSearch(node, g, threshold){
    f = g + heuristic(node)
    if (f>threshold):
        return f
    if (node==Goal):
        return TRUE
    integer min=MAX_INT
    foreach (tempnode in nextnodes (node))do
        temp = DFSearch(tempnode, g+cost(node,
tempnode), threshold)
        if (temp==FOUND):
            return TRUE
        if (temp<min):
            min=temp
    return min
}//end search function
source: (Wikipedia, 2022)

```

We chose to increase the depth at each iteration by 1. Typically the search depth will be expanded by the smallest f value that was too large for the last depth, but in the case of the Rubik's cube, we believe that it is more efficient to always increment the depth by 1 since the vast majority of all cube states will take 17 or 18 moves to solve (Kociemba, 2010).

Rubik's Cube Description and Terminology



- Cubie: there are 20 moveable subcubes, called “cubies”.
- Corners: There are 8 corner cubies, each with three different coloured faces.
- Edges: There are 12 edge cubies, each with two different coloured faces.
- Face: each side of the Rubik's cube has 9 squares. On each face, 4 of the squares are part of edge pieces, 4 are part of corner pieces, and there is one centre square. The six faces are called, front, back, left, right, up and down.
- Twists: A Rubik's Cube face can be turned 90 degrees clockwise or counterclockwise, or 180 degrees. As such, there are 18 possible moves at any given state.

Indexing with Lehmer Codes

In order to access entries in the pattern database, we need an efficient algorithm to generate sequential integers that represent every cube state (for certain subsets of the cube). We decided to implement the databases as arrays that store the number of moves required to solve the cube state associated with the index code. We used Lehmer codes, a lexicographic numbering method to generate a code representing the cubies permutation. Then we converted that code to base-10. The Lehmer code for corners will output a number between 0 and

40319, since there are 40320 (8!) ways to permute 8 objects. Here is an example of what that looks like for the 20160th permutation of the 8 corners.

Permutation	Order
(4 0 1 2 3 5 6 7)	20160

The corners also have an orientation that generates another code. Each corner can have three different orientations (enumerated 0, 1, 2), so this list can simply be treated as a base 3 number. Since a single corner cannot be rotated on the Rubik's cube, the orientation of 7 corners determines the orientation of the 8th. Therefore every permutation of corners has 3^7 possible orientations of the corners. The codes are combined to a single code that is the permutation code multiplied by 3^7 plus the orientation code (converted to base 10). A similar process is applied for the edges.

Since every single state will have multiple Lehmer codes calculated for it, this must be a fast algorithm. Korf actually also created a linear time algorithm for this. The algorithm sets bits for numbers already encountered in the permutation, and calculates the Lehmer code with bit shifts and a precomputed table of how many 1s are in a certain range of binary numbers. It is very fast.

Algorithm 2: Lehmer Code

```
function factorial(n){
    return n <= 1 ? 1 : n * factorial(n-1)
} //end factorial function

function LehmerCode(n){
    onesInBinaryNumb = [1:n]
    foreach(i from 0 to 2) do
        onesInBinaryNumber[i] = bits.count()
    factorialList = { factorial(2), factorial(1), factorial(0) }
    permute = {1, 1, 0}
    lehmer[0] = permute[0]
    foreach(i from 1 to permute.size()-1)do
        visited[permute.size() - 1 - permute[i]] = 1
        totOnesCount = onesInBinaryNumber[i]
```

```
        lehmer[i] = perm[i] - totOnesCount
    foreach(i from 0 to n)do
        index[i] = convertToDecimal(i)
    return index
} //end LehmerCode function
source: (Botto, 2019)
```

Example calculation: To compute the index of a given permutation, we take the associated digit of that permutation's Lehmer code and subtract from it the number of permutation elements to its left that are less than it (Wikipedia, 2022). For example if we have the permutation (1 2 0) that gives us a Lehmer code of 110, and converting 110 to base-10 gives us 2, so 120 is the second permutation of the set {0 1 2} if we lexicographically order it.

Pattern Databases

As Korf suggests, we implemented three databases, one for the eight corners and another two for the edges. The database will return the minimum number of moves to solve certain subsets of the Rubik's cube. To solve only the 8 corner cubies, the database stores all 88,179,840 different corner states the cube can take on as well as the number of moves needed to solve each state (Korf, 2005).

In Korf's original paper he suggests edge databases for 2 groups of 6 edges. Due to advances in computer hardware and memory, we expanded the databases to include subsets of 7 edges. There are over 500 million ways 7 edges of the cube can be permuted and oriented, so these two databases are quite a lot larger than the corner PDB.

The corner PDB was generated with a simple BFS. The number of states is small enough that little care is required to populate the database in reasonable time.

The edge PDB uses IDDFS to reach all the edge states. There are too many states to keep in memory, so the extra cost of searching the same states many times must be afforded. Each edge database took around 16 hours to generate.

We also generated a fourth database, which stores the number of moves required to orient all 12 edges. This was not in the original Korf paper, but suggested by Ben Botto in an article he wrote (see references). This PDB was trivial to generate with only 2048 states, but we have doubts about its usefulness. Edges can always be oriented in 7 moves or fewer, and we do not think there are many states where edge orientation takes more moves than solving either of the 7 edge groups.

Our implementation takes advantage of reduction of problem size using symmetry to significantly reduce the amount of work there is to be done. We leverage symmetry such that no matter the state of the scramble, there will always be the same set of face rotations available to us (Culberson, Schaeffer, 1998). Each twist moves four edges and four corner cubies, therefore the same sequence of twists that result in a solution may appear in more than one pattern database (Korf, Felner, 2002). This is why Korf's algorithm uses multiple pattern databases and picks the maximum value among all the estimates generated as its heuristic; this guarantees we never overestimate the solution cost.

For instance, when checking the PDBs, if there are 6, 3, or 4 moves needed to solve each PDBs subset of the cube, then the A* heuristic used will be 6. This is the lower bound estimate of the number of moves a given scrambled state is from the solution state. We want the lower bound to be as large as possible, so nonoptimal moves can

be pruned sooner and lower the overall search time.

Methodology

The question we wanted to answer was “will the number of available PDBs affect the runtime of Korf's algorithm”.

Originally, we had planned on comparing Korf's algorithm to other algorithms such as Thistlethwaite's and uninformed searches such as BFS with respect to space & time required and difficulty of implementation. We had also planned on comparing how their performance (Korf's and Thistlethwaite's) would have been affected by the quantity of PDBs present, however, due to time constraints, we had to change our plans. We also learned that due to system and time constraints it is unrealistic to do an uninformed/brute force search on a Rubik's cube, the algorithm might never complete its operations due to the sheer number of permutations.

Finally we decided to implement an algorithm that takes in as input a given scramble (a string of moves), and solves the Rubik's cube using IDA* search and three pattern databases. We repeated the tests multiple times, recorded the run times and took the average to even out any anomalies.

Experimental Setup

We configured our solver to compile on Linux using g++. We implemented the solver using C++ for performance reasons and our preexisting familiarity with the language.

We ran our tests on a desktop computer equipped with a AMD Ryzen 5 3600x @ 4.25Ghz and 32GB of RAM and a laptop

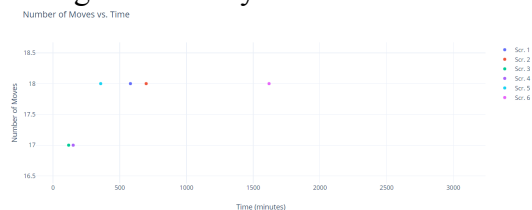
with an Intel I5-9300H @ 2.40Ghz and 16GB of RAM which were both running a virtual machine on Ubuntu 20.04.

On the desktop, we chose to run 3 instances of the solver in parallel to help boost efficiency as running one instance of the solver took multiple hours. Since the solver takes so long to solve a cube, it severely hampers our ability to make conclusions about runtime. We were limited in the amount of cubes we could solve, so we are severely lacking data points. Nevertheless, we will consider the data that we do have.

Performance Analysis and Results

All times are in realtime			
	No PDB	1 PDB	3 PDBs
Scramble	L2 U2 R2 D2 B D2 R2 F2 U2 F' D R F2 R' F2 L2 B2 D B' F'		
Solution	B2 D F U F' U' L2 U' F2 D' F' B' U' R D' R B U2		
Time	DNF	DNF	580m, 15.876s
Scramble	F' L2 U R2 U' R2 U' B2 D B2 U B2 F R' B' D2 F2 U B' F		
Solution	U2 F' B U' B2 D2 R2 B R' F U' R2 U L' U2 L' U F'		
Time	DNF	DNF	698m, 54.435s
Scramble	L' B' F2 L2 B2 U2 R2 D R2 D' B2 R2 B' R2 D B' R F' L		
Solution	R D B' L B2 L2 D' B' D2 L2 F L' U' R' L' U2 F'		
Time	DNF	DNF	117m, 22.506s
Scramble	F2 L' D2 R B2 R2 U2 L' D2 B2 D2 R2 U R' F' D' U B' R U' R2		
Solution	D' F' R2 L2 D B' U' R2 F2 L' F' R' D B U' L2 F2		
Time	DNF	DNF	151m, 53.919s
Scramble	F D F B R D' F' L' B' L2 D' R2 D2 R2 F2 L2 D F2 L2 U2 L2		
Solution	R2 B' L B' L' D L U2 B L F D2 R' B' D2 R F B2		
Time	DNF	DNF	357m 19.437s
Scramble	L2 D U' L2 U R2 F2 D2 R2 L F2 D' R' B R2 F2 D L F'		
Solution	F2 L' U2 L' U' D R F R2 L U2 D' F U2 L' U' F2 L'		
Time	DNF	DNF	1619m, 59.157s

As shown in our table, the instances running with no and 1 PDB were unable to finish, leaving us with only one source of data.



Please click [here](#) to expand the table and chart

As seen above in the chart, the number of moves required to solve the cube seems to have a strong relation to the time taken. 17 move solutions can be solved relatively quickly, while 18 moves or more had significantly longer runtimes. Although rare, there are cubes that require 20 moves to solve. Based on how long 18 move solutions took to find, we think attempting a 20 move cube would take days to find the solution.

We did notice that there were times when the solver would get lucky and find a solution quickly, or it could go down the wrong branch and take an unreasonable amount of time to solve.

All of the tests were done on the desktop computer as the laptop was simply too slow so we decided to terminate the solver prematurely.

As a result of our experiments, we found that even with 3 PDBs, our realtime average solving time was 587 minutes, or 9.78 hours. In cases where we used no PDB or had 1 PDB, the solver would take so long (>1 day) such that we had to end the solver unsuccessfully so that we could allocate resources to other computations. In one example, an instance of the solver running on the laptop was stuck on depth 15 for roughly 16 hours before we decided to stop it.

Overall, we found that the presence of PDBs helped improve the speed of the solver, which is the purpose of computing PDBs. However, due to the computation time, we were unable to determine what the magnitude of improvement would be if the solver was run with 2 PDBs versus 3 PDBs for example.

Conclusions

To find the optimal solution to a Rubik's cube puzzle we implemented Korf's algorithm which combines IDA* search with pattern databases as a heuristic to guide the algorithm as it traverses the states. The heuristic helps discover if it is getting further away from or closer to the solution. Although generating the databases was time consuming, our experiment was able to find optimal solutions. That is, solve the Rubik's cube in the shortest possible number of moves, which over all states is at most 20. Experimenting with one versus three pattern databases, we found that only using one took so long for our computer systems, that we had to abandon the execution and acquiesce that a solution could not be found with the given resources available to us. Using all the pattern databases did result in a solution every time. In the future, we can improve this experiment by building even more powerful PDBs (expand to 8 edge groups for example). Additionally, we can experiment with different search algorithms and data structures to improve our results. Although this implementation was for amusement, this project helped us learn a lot about different search algorithms and how they can be applied to solve puzzles.

References

- Korf, R. E. (2005). Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. *Computer Science Department University of California, Los Angeles*. Retrieved August 18, 2022, from <https://www.cs.princeton.edu/courses/archive/fall06/cos402/papers/korfrubik.pdf>.
- Botto, Ben. (2019, May 26) *Sequentially Indexing Permutations: A Linear Algorithm for Computing Lexicographic Rank*. Medium. Retrieved August 18, 2022, from <https://medium.com/@benjamin.botto/sequentially-indexing-permutations-a-linear-algorithm-for-computing-lexicographic-rank-a22220ffd6e3>
- Botto, Ben. (2020, May 8) *Implementing an Optimal Rubik's Cube Solver using Korf's Algorithm*. Medium. Retrieved August 18, 2022, from <https://medium.com/@benjamin.botto/implementing-an-optimal-rubiks-cube-solver-using-korf-s-algorithm-bf750b332cf9>
- Culberson, J. C., & Schaeffer, J. (1998). Pattern databases. *Computational Intelligence*, 14(3), 318–334. <https://doi.org/10.1111/0824-7935.00065>
- Korf, R. E., & Felner, A. (2002). Disjoint pattern database heuristics. *Artificial Intelligence*, 134(1-2), 9–22. [https://doi.org/10.1016/s0004-3702\(01\)00092-3](https://doi.org/10.1016/s0004-3702(01)00092-3)
- Wikimedia Foundation. (2022, July 10). *Iterative deepening A**. Wikipedia. Retrieved August 18, 2022, from https://en.wikipedia.org/wiki/Iterative_deepening_A*
- Wikimedia Foundation. (2022, June 28). *Lehmer Code*. Wikipedia. Retrieved August 18, 2022, from https://en.wikipedia.org/wiki/Lehmer_code